


# Amrita School of Engineering
# Bengaluru Campus

## Department of Electronics & Communication Engineering

## 23ECE102

## Problem Solving and Algorithmic Thinking

## B.Tech I Semester ECE and EAC

## Lab Manual

## Vision and Mission of the Institute

| | | |
|---|---|---|
| Vision | | To be a global leader in the delivery of engineering education, transforming individuals to become creative, innovative, and socially responsible contributors in their professions. |
| Mission | M1 | To provide best-in-class infrastructure and resources to achieve excellence in technical education |
| | M2 | To promote knowledge development in thematic research areas that have a positive impact on society, both nationally and globally |
| | M3 | To design and maintain the highest quality education through active engagement with all stakeholders – students, faculty, industry, alumni and reputed academic institutions |
| | M4 | To contribute to the quality enhancement of the local and global education ecosystem |
| | M5 | To promote a culture of collaboration that allows creativity, innovation, and entrepreneurship to flourish |
| | M6 | To practice and promote high standards of professional ethics, transparency, and accountability |

## Vision and Mission of the Department

| | | |
|---|---|---|
| Vision | | To provide a value-based learning environment for producing engineers with a blend of technical skills, moral values and leadership qualities in the field of Electronics, Communication and Computing channelized towards technological advancement to cater to the needs of the industry and the society. |
| Mission | M1 | Achieving excellence in teaching and learning with an emphasis on fundamental knowledge and hands-on exposure to match the state-of-the-art in technology. |
| | M2 | Providing an environment for core competency development and enhancing quality research in emerging areas. |
| | M3 | Facilitating professional growth to the students for higher education and career in industry and academia |
| | M4 | Imbibing the essence of human values, ethics and professional skills to sustain socio-economic development |

AMRITA VISHWA VIDYAPEETHAM

## AMRITA SCHOOL OF ENGINEERING

**BENGALURU**

# PRACTICAL RECORD BOOK

**Academic year: 20_ - 20_ (ODD/EVEN Semester)**

**Department:** .....

**Name:**.....

**Reg. No:**.....

**Sem./ Section:**.....

**Branch:**.....




# AMRITA SCHOOL OF ENGINEERING

## BENGALURU

**Department:**

Certified that this is the bona fide record of work done by Mr./Ms.....

Register No.....

in the .....

Laboratory of this Institution, as prescribed by the Amrita Vishwa Vidyapeetham for the .....Semester B.Tech. During the academic year 20....- 20... (ODD/EVEN Semester).

**Faculty In Charge** **Chairperson**

Submitted for the .....semester B.Tech. Examination of the Amrita Vishwa Vidyapeetham conducted on ...../..../20.....

**Examiner 1** **Examiner 2**

# CONTENTS

| Exp. No. | Date | Title of the Experiment | Record Marks | Experiment Marks | Faculty sign |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |
| 6 | | | | | |
| 7 | | | | | |
| 8 | | | | | |
| 9 | | | | | |
| 10 | | | | | |
| 11 | | | | | |
| 12 | | | | | |
| 13 | | | | | |
| 14 | | | | | |
| 15 | | | | | |

# Course Plan

## 23ECE102/ Problem Solving and Algorithmic Thinking
## I Semester ECE and EAC

### Course Objectives

- To provide insight into computational logic
- To introduce the fundamentals of computational thinking
- To introduce computational approach to problem solving

### Course Outcomes:

- CO1: understand the concepts of computational logic
- CO2: develop algorithmic thinking
- CO3: identify algorithms and their suitability
- CO4: apply algorithms to solve a problem

### CO-PO Mapping:

| PO/PSO / CO | PO1 | PO2 | PO3 | PO4 | PO5 | PO6 | PO7 | PO8 | PO9 | PO10 | PO11 | PO12 | PSO1 | PSO2 | PSO3 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| CO1 | 2 | | | | | | | | | | | 3 | | | |
| CO2 | 2 | | | | | | | | | | | 2 | | | |
| CO3 | 2 | 2 | | | | | | | | | | 2 | | | |
| CO4 | 3 | 2 | | | | | | | | | | 2 | | | |

### Experiment Contents:

1. Familiarization with flowgorithm.
2. Visualization of logical flow in flowgorithm using addition and subtraction of two numbers.
3. Exposure to various formatting methods using problems on addition, subtraction.
4. calculation of area of circle and identification of odd even numbers.
5. Arithmetic operations on vectors and matrices.
6. Solving Quadratic equations and generation of Fibonacci numbers
7. Modelling Simple resistive circuits
8. Use of arrays in solving problems.
9. Familiarization with strings.
10. Searching (linear and binary)
11. Sorting (bubble sort, insertion sort and selection sort)
12. Modelling circuits with dependent sources.

## System Design:

Example:

1. Develop logic/flow chart/algorithm for a multifunctional calculator.
2. Model different circuit topologies with dependent sources and analyse resulting voltage and current sources.

## Weekly Progress Monitoring for System-Level Design Project

The progress of the system-level design project will be monitored on a weekly basis through weekly mentor meetings. Each week, student teams are required to submit a brief progress outlining the tasks completed, challenges faced, and plans for the upcoming week. These discussions help track incremental development and identify any roadblocks early. Additionally, weekly review meetings, mentors will provide guidance, assess progress against the project timeline, and ensure alignment with course objectives. This structured, iterative approach encourages continuous engagement, timely problem-solving, and steady advancement towards the final project deliverables.

## Guidelines to Teachers

1. **Consistency in Delivery:** Ensure uniformity in delivering the course content by strictly following the approved syllabus and lab manual.
2. **Student Engagement:** Encourage active participation during lab sessions.
3. **Monitoring Progress:** Maintain regular oversight of students' progress, especially for system-level design projects. Provide timely feedback and support to help students overcome challenges.
4. **Assessment and Evaluation:** Evaluate students based on their practical skills, project work, and understanding of concepts as per the guidelines. Use rubrics to ensure fairness and transparency.
5. **Documentation:** Ensure that students maintain proper lab records, including observations, results, and reflections. Review and verify these records periodically.

## Guidelines to Students

1. **Preparation Before Lab Sessions:**
    - Students should prepare for the upcoming experiment and come prepared for the lab.
    - For software experiments, students can run the provided sample codes and verify the outputs prior to starting their own coding tasks for the day's experiment.
2. **Timely Attendance and Participation:**
    - Arrive on time with all necessary materials and pre-lab work completed to ensure full participation in the lab activities.
    - Actively engage in all experiments, discussions, and demonstrations during the lab session.
3. **Lab Record Maintenance:**
    - Maintain a well-organized and updated lab notebook containing experiment objectives, procedures, observations, results, and conclusions.
    - Submit the lab records as per the instructor's schedule for review and grading.
4. **Teamwork and Collaboration:**
    - Collaborate effectively with lab partners, sharing responsibilities and knowledge while respecting individual contributions.
    - Seek help from instructors or mentors whenever facing difficulties.
5. **Safety and Discipline:**
    - Follow all safety guidelines strictly while handling equipment and conducting experiments.
    - Maintain discipline and cleanliness in the lab at all times.
6. **Weekly Project Work:**
    - For system-level design projects, prepare weekly progress reports and be ready to present updates during scheduled review sessions.

# Experiment – 1
# Familiarization with flowgorithm

## Aim: To understand the Flowgorithm tool and its application for Problem solving.

## Tools required: Flowgorithm App.

## Theory:

### Introduction to Flowgorithm:

Flowgorithm is a visual programming tool designed to help beginners learn programming concepts and logic. It allows users to create flowcharts and pseudocode, enabling them to understand the logic behind algorithms and problem-solving processes without worrying about the complexities of syntax in a specific programming language. Flowgorithm is particularly popular in educational settings and is often used in computer science courses to teach introductory programming concepts.

**Key features of Flowgorithm include:**

**Visual Interface:** Flowgorithm provides a user-friendly, drag-and-drop interface that allows users to create flowcharts and pseudocode easily. This graphical representation helps learners visualize the control flow and the sequence of steps in an algorithm.

**Cross-platform:** Flowgorithm is available for Windows, macOS, and Linux, making it accessible to a wide range of users on different operating systems.

**Code Generation:** Though Flowgorithm emphasizes visual representation, it also allows users to convert their flowcharts and pseudocode into several programming languages like C, C++, Java, Python, and more. This feature is helpful for users who want to transition from visual programming to writing code in a specific language.

**Educational Tool:** Flowgorithm is often used as an educational tool to introduce programming concepts to students in a simplified manner. It can be a stepping stone for beginners before they move on to more complex programming languages and environments.

**Community and Resources:** Flowgorithm has an active user community, and several resources, tutorials, and sample programs are available to assist learners in understanding programming concepts better.

While Flowgorithm is excellent for beginners and educational purposes, it might not be suitable for professional software development due to its limited scope and focus on introductory concepts. Experienced programmers generally prefer integrated development environments (IDEs) and text-based code editors that offer more advanced features and support for large-scale software projects.

In summary, Flowgorithm is a valuable tool for individuals who are new to programming and seek to understand the basics of algorithms and problem-solving through a visual and intuitive approach. It serves as an excellent starting point before diving into full-fledged programming languages and development environments.

## Data Types

### Integer Data Type

The Integer data type is one of the most commonly used types in programming. An integer can store a positive or negative whole number, but can't store fractional values. So, it can store values such as 5, 42, 1947, but can't store numbers such as 3.2, 4.5, etc...

If a number with a fractional value is stored into a integer, the fractional value will be discarded.

Hence, if 3.2 is stored into an integer, it will only retain 3.




The Variable Watch Window displays integers in blue.

### Real Data Type

The Real data type can store any number - both whole numbers and ones with fractional values. In many languages, this is called a "double" after the implementation standard known as "double-precision floating point".




The Variable Watch Window displays reals in purple.

### String Data Type

The String data type is used to store any textual data. This includes words, letters, or anything else you would send in a text message. In programming, the text is delimited with double quotes.

For example: "CSU, Sacramento", "computer", and "Year 1947" are all strings.

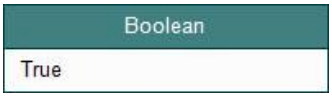
The Variable Watch Window displays strings in red.




### Boolean Data Type

The Boolean Data Type can store either "true" or "false". These are the basis of decision making in a computer program.

The Variable Watch Window displays Booleans in teal.




### Table 1: Summary Chart

| Data Type | Notes |
|---|---|
| `Boolean` | Stores either Boolean true or false |
| `Real` | Stores a real number. |
| `Integer` | Stores an integer number. |
| `String` | Stores textual data. |

## Identifiers

### Naming Rules

Any time you define a function or variable, it is given a unique name called an "identifier". To prevent identifiers from being confused with other items in an expression, they must follow a ***naming convention***. Every programming language has one and they are fairly consistent from language to language.

In Flowgorithm, identifiers must adhere to the following rules:

- They must start with a letter.
- After the first letter, the identifier can contain additional letters **or** numbers.
- Spaces are **not** allowed.

They cannot be key words or words already defined by Flowgorithm (please see below)

**Also note:**

- Languages such as Visual Basic and C also allow the underscore character "_".
- Flowgorithm, however, does not allow it.

Identifiers are not case-sensitive.

### Examples

| Valid Identifiers | Notes |
|---|---|
| x | Identifiers can be single letter. |
| noun2 | Numbers are allowed after the first letter |

## Keywords

### Overview

Programming languages often make use of words such as "class", "public", and "if" for readability. Often, these overlap with the naming convention used by identifiers. So, in these cases, the word is considered "reserved" and cannot be used for Identifiers. Many programming languages also predefine functions and other constants and cannot be used.

### Reserved Words

Flowgorithm only has a few reserved words that are used in **expressions**.

| | | | | | | |
|---|---|---|---|---|---|---|
| `and` | `false` | `mod` | `not` | `or` | `pi` | `true` |

## Data Type Keywords

To prevent confusion, the system doesn't allow identifiers to use the name of the **Data types**.

| | | | |
|---|---|---|---|
| boolean | integer | real | string |

## Illegal Keywords (used in functions)

Flowgorithm does not permit the names of **intrinsic functions** to be used.

| | | | | |
|---|---|---|---|---|
| abs | cos | random | tan | tostring |
| arccos | int | sgn | tochar | toreal |
| arcsin | len | sin | tocode | |
| arctan | log | size | tofixed | |
| char | log10 | sqrt | tointeger | |

## Reserved for Future Use

Future versions of Flowgorithm may expand the number of intrinsic functions. The following were reserved if, someday, they are added.

| | |
|---|---|
| arccosh | cosh |
| arcsinh | sinh |
| arctanh | tanh |

# Operators

## About

The notation used for comparisons logical operators, etc... vary greatly between different programming languages. Flowgorithm supports the symbols used in mathematics (using Unicode values) as well as the two major families of programming languages. The "BASIC-family" contains English keywords and operators. The "C-family" (which includes C, Java, C#) is far more symbolic.

Since mathematics and two major language families are supported, there are redundant operators. Flowgorithm considers the redundant operates as the same - and any set can be used (or all of them). This allows the student to use the operators that match the language they plan to learn later.

| Operator | C Family | BASIC Family | Mathematics (Unicode) |
|---|---|---|---|
| Equality | == | = | = |
| Inequality | != | <> | ≠ |
| Less Than or Equal | <= | <= | ≤ |
| Greater Than Or Equal | >= | >= | ≥ |
| Logical Not | ! | not | ¬ |
| Logical And | && | and | ∧ |
| Logical Or | \|\| | or | ∨ |
| Multiply | * | * | × |
| Divide | / | / | ÷ |
| Modulo | % | mod | |

Flowgorithm also adds a few unique Visual Basic operators since if they have helpful, clearly defined, semantics

| Name | Basic Family | Mathematics (Unicode) | Notes |
|---|---|---|---|
| String Concatenation | & | | C# and Java use the ambiguous "+" operator for addition and concatenation. |
| Exponent | ^ | ↑ | Note: this is not a typical mathematical symbol, but is often used when superscripts cannot be used. |

In Java and C#, the + operator is used for both string concatenation and addition. This can be quite confusing given the rather complex semantics. In Flowgorithm, addition will only work with numbers. The ampersand & is used for concatenation. Also, C# and Java lack an exponent operator - instead relying their respective Math classes. Flowgorithm uses the Visual Basic ^.

## Precedence

The following are the precedence levels from high (evaluated first) to low.

| Level | Name | Operators | Notes |
|---|---|---|---|
| 8 | Unary | `-  !  not  ¬` | In Visual Basic, "not" precedence level is far lower - above "and", but below all relational operators. |
| 7 | Exponent | `^  ↑` | The exponent operator does not exist in C# or Java. |
| 6 | Multiply | `*  ×  /  ÷  %  mod` | Division will always be high-precision (floating point) |
| 5 | Addition | `+  -` | In Flowgorithm, "+" will only work with numbers. |
| 4 | Concatenate | `&` | |
| 3 | Relational | `>  <  >=  ≥  <=  ≤  ==  =  !=  <>  ≠` | |
| 2 | Logical And | `and  &&  ∧` | |
| 1 | Logical Or | `or  ||  ∨` | |

## Examples

| Expression | Result | Notes |
|---|---|---|
| `1 + 3 ^ 2` | `10` | |
| `10 * 2 + 5 * 6` | `50` | 10 * 2 and 5 * 6 have higher precedence than addition. The addition is done last. |
| `7 * (4 - 1)` | `21` | Parenthesis are used for subexpressions, which are evaluated as a whole. |
| `6 / 3 * 2` | `4` | In mathematics, multiplication and division have the same precedence levels. So, they are evaluated left-to-right. The "PEMDAS" acronym, used in high-school, is a tad misleading. |
| `10 mod 3` | `1` | Modulo math gives the remainder from division |
| `10 % 3` | `1` | Same expression, but using the C-Family operator |

## Intrinsic Functions

### Mathematics

| Function | Description |
|---|---|
| `Abs(n)` | Absolute Value |
| `Arcsin(n)` | Trigonometric Arcsine |
| `Arccos(n)` | Trigonometric Arccos |
| `Arctan(n)` | Trigonometric Arctangent |
| `Cos(n)` | Trigonometric Cosine |
| `Int(n)` | Integral (whole value) of a real number |
| `Ln(n)` | Natural Log |
| `Log(n)` | Natural Log (same as Ln) |
| `Log10(n)` | Log Base 10 |
| `Sgn(n)` | Mathematical sign (-1 if ***n*** is negative, 0 if zero, 1 if positive) |
| `Sin(n)` | Trigonometric Sine |
| `Sqrt(n)` | Square Root |
| `Tan(n)` | Trigonometric Tangent |

### Strings

| Function | Description |
|---|---|
| `Len(s)` | Length of a string |
| `Char(s, i)` | Returns a character from the string ***s*** at index ***i***. Characters are indexed starting at 0. |

### Data Type Conversion

| Function | Description |
|---|---|
| `ToChar(n)` | Convert a character code ***n*** into a character. |
| `ToCode(c)` | Convert a character ***c*** into a character code (integer). |
| `ToFixed(r, i)` | Convert real number ***r*** to a string with ***i*** digits after the decimal point. This function is useful for currency. |
| `ToInteger(n)` | Convert a string to an integer |
| `ToReal(n)` | Convert a string to an real |
| `ToString(n)` | Convert a number to a string |

## Other

| Function | Description |
|---|---|
| `EOF()` | Returns true of the end of the file was reached. This is used with files opened for reading. |
| `Random(n)` | A random number between 0 and (***n*** - 1) |
| `Size(a)` | The size (number of elements) in an array |

## Built-in Constants

Flowgorithm defines three commonly used constants. The constants true and false are often used to initialize Boolean variables.

The constant pi is commonly used in mathematics.

| Constant | Notes |
|---|---|
| `true` | Boolean True |
| `false` | Boolean False |
| `pi` | Mathematical Pi. Approximately 3.1415. |
| `π` | Mathematical Pi. Same as the above. |

# Assignment Shape

## Default Appearance

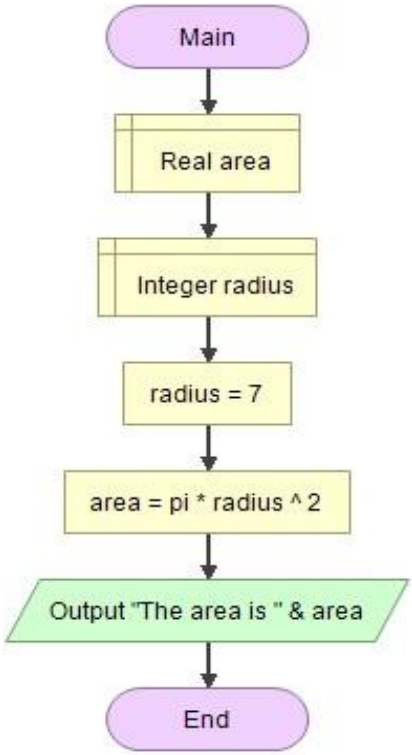



## What it Does?

The Assignment shape is used to store the result of a calculation into a variable. This is one of the most common tasks found in programs.

## Example

The example, to the right, declares two variables: area (which stores real numbers) and radius (which stores integers). It then uses an Assignment Statement to set the 'radius' to 7. Finally, it computes the area of a circle and stores the result in 'area'.



# Breakpoint Shape

## Default Appearance



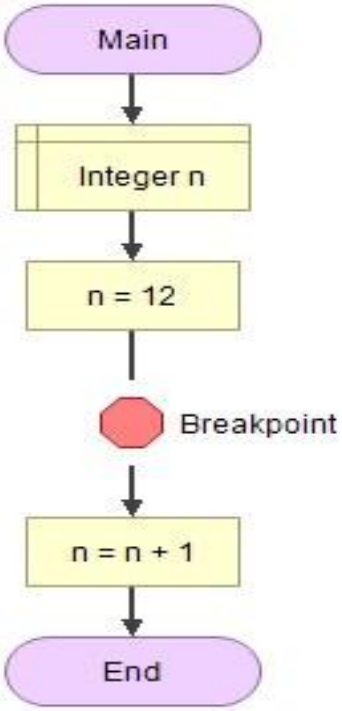

## What it Does

The Breakpoint Shape temporality halt the execution of the program. This is useful both for debugging programs and for demonstrations. Most professional software development applications have some form of the breakpoint.

## Example

The example, to the right, creates a variable called 'value' and assigns it an initial value of 12. The program then encounters a breakpoint shape - which still halt execution.

This will allow the user to see the current value of 'n' in the Variable Watch Window. Once execution is continued, the second assignment shape is executed.



# Call Shape

## Default Appearance



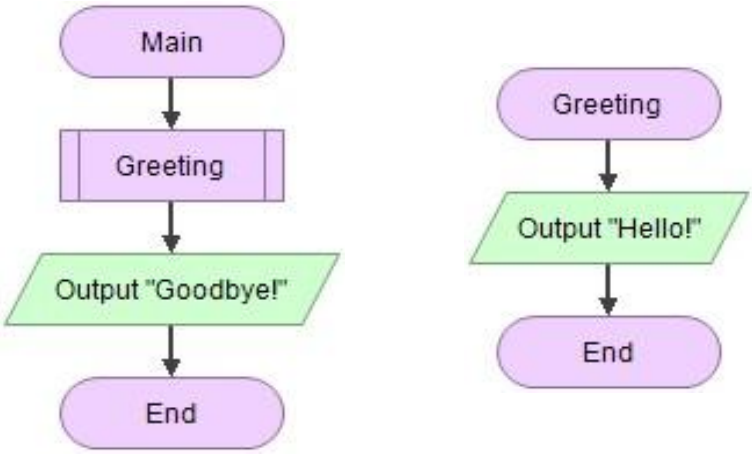

### What it Does

A Call Statement transfers control to a function. Information being passed into the function are called 'arguments'.

## Example

The following example uses the Call Shape to execute a function called 'Greeting'.

When the program executes, the first shape will call the Greeting Function. After it outputs "Hello!", it will return and the Main Function and it will output "Goodbye!".



# Comment Shape

## Default Appearance



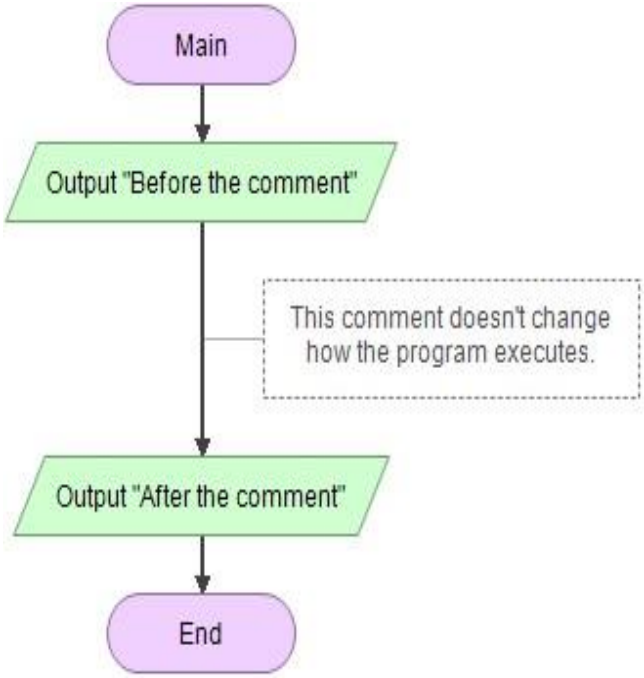

## What it Does

Comments don't affect how your program runs. They are used to include documentation about the program for other programmers.

These can include: the logic of a loop, known issues, changes made, future changes, etc...

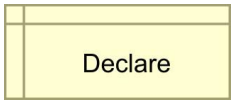
## Example

The example, to the right, contains a comment between two Output Shapes. It has no affect on the program.



## Declare Shape

### Default Appearance

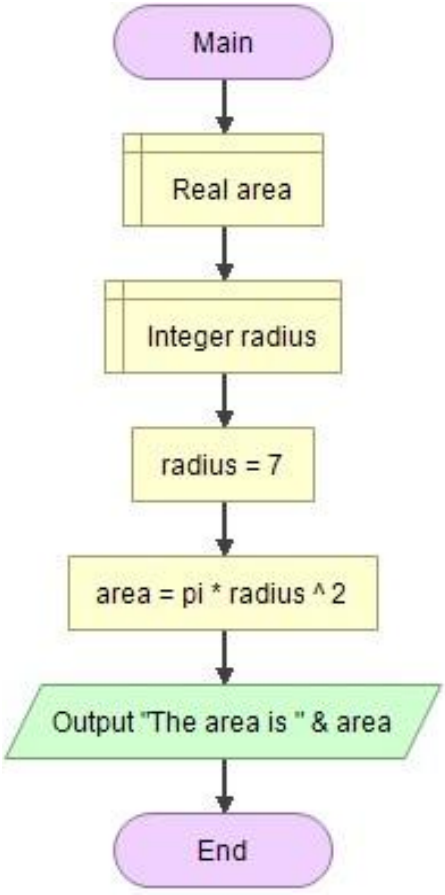



### What it Does

A Declare Statement is used to create variables and arrays. These are used to store data while the program is running. You can declare multiple variables by separating the names with commas.

### Example

The example, to the right, declares two variables: area (which stores real numbers) and radius (which stores integers). It then uses an Assignment Statement to set the 'radius' to 7. Finally, it computes the area of a circle and stores the result in 'area'.



### Do Shape

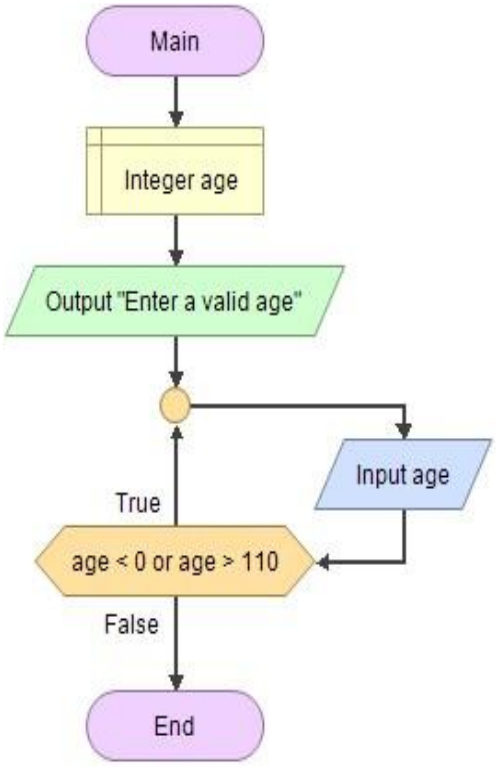
### Default Appearance

### What it Does

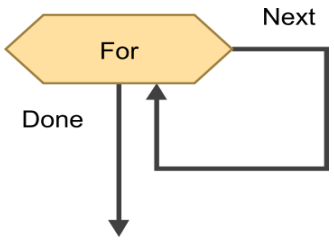
A Do Loop is similar to a While Loop except that the block of statements is executed at least once before the expression is checked.

### Example

The example, to the right, shows a Do Statement that accepts only a valid age as input. It will loop while the 'age' variable is less than 0 or greater than 110.



**For Shape**
**Default Appearance**

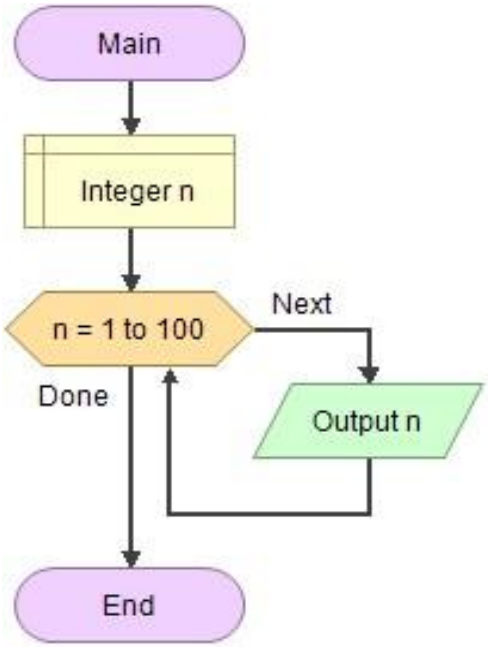



### What it Does

For Loops increment a variable through a range of values. This is a common, useful, replacement for a While Statement.

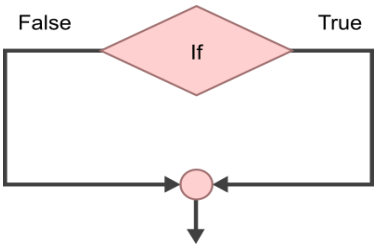
## Example

The example, to the right, prints the numbers from 1 to 100. The loop executes 100 times. The value of 'n' starts at 1 and increases by 1 each time the loop executes. The loop ends when 'n' reaches 100.



## If Shape

### Default Appearance



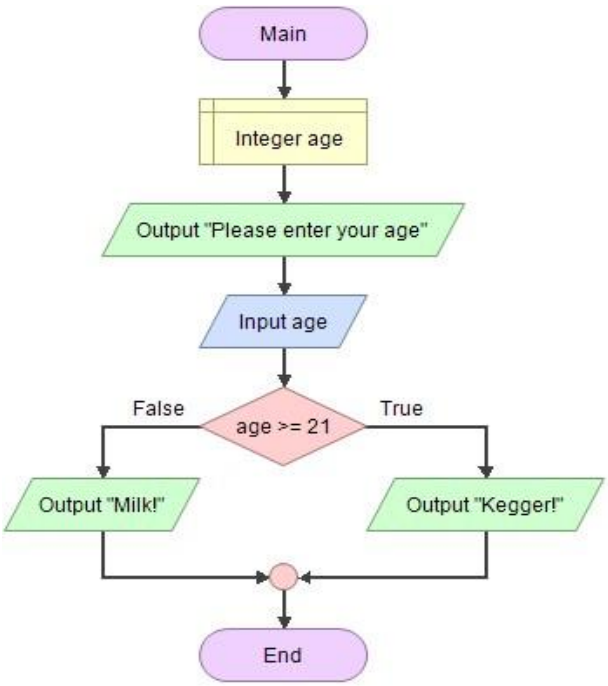

### What it Does

An If Statement checks a Boolean expression and then executes a true or false branch based on the result.

### Example

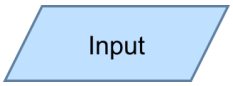
The example, to the right, declares an integer called 'age'. It then reads the age from the keyboard.

Finally, an If Statement checks if the age is greater than or equal to 18. Based on this, it either takes the false branch and displays "Sorry, not yet", or takes the true branch and displays "Go vote!".



## Input Shape

### Default Appearance

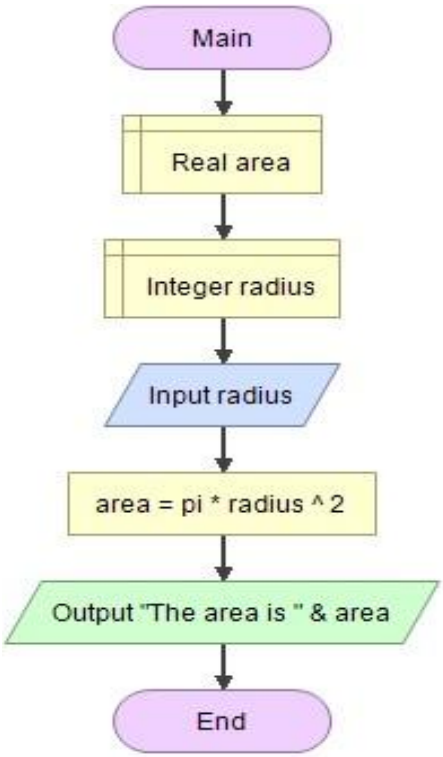



### What it Does

An Input Statement reads a value from the keyboard and stores the result in a variable.

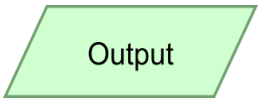
### Example

The example, to the right, creates two variables: 'area' and 'radius'. It then uses an Input Statement to read the radius from the keyboard. A final Output Statement then displays the result.



## Output Shape

### Default Appearance

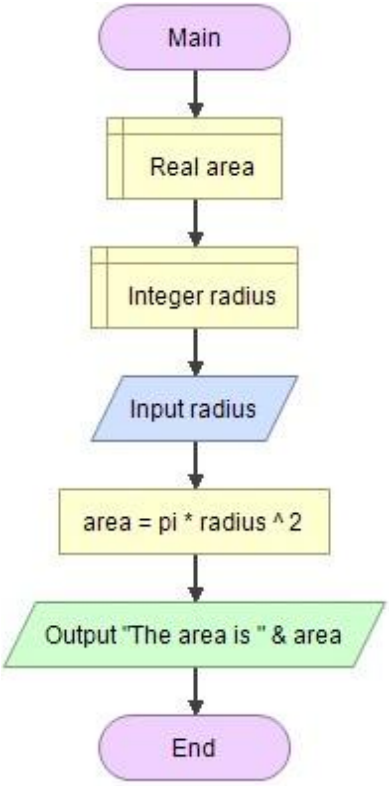



### What it Does

An Output Statement evaluates an expression and then displays the result on the screen.

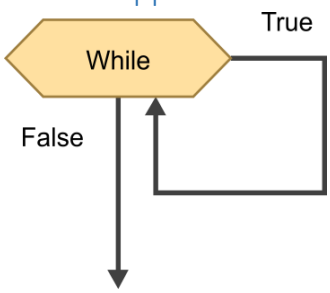
### Example

The example, to the right, creates two variables: 'area' and 'radius'. It then uses an Input Statement to read the radius from the keyboard. A final Output Statement then displays the result.



## While Shape

### Default Appearance

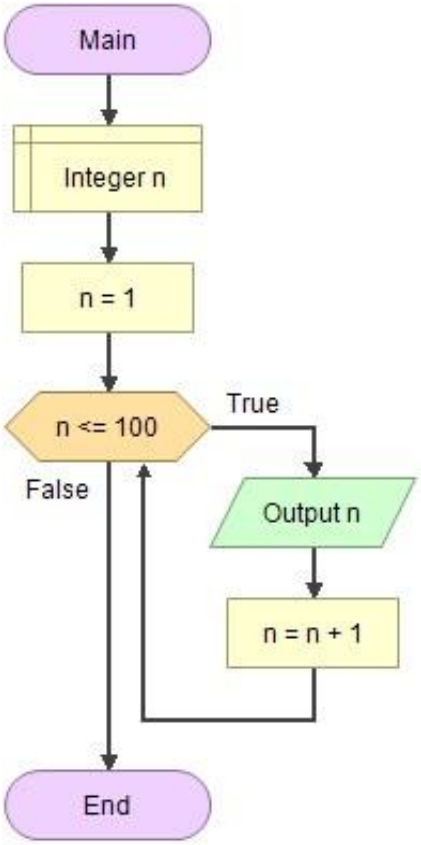



### What it Does

A While Loop evaluates a Boolean expression and then, if true, executes a block of statements. After the statements are executed, the While Statementt rechecks the expression. When the expression is false, the loop ends.

### Example

The example, to the right, prints the numbers from 1 to 100. The assignment statement "n = n + 1" increments the variable 'n' by 1 for each iteration of the loop.



## Forward Shape

### Default Appearance




### What it Does

The Forward Shape is used with the Turtle Graphics window. It moves the "turtle", an pen, forward a specified distance. If the pen is down, a line will be drawn. When up, the turtle moves without creating any mark.

## Home Shape

### Default Appearance

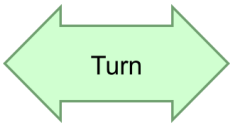



### What it Does

The Home Shape is used in the Turtle Graphics window. It moves the turtle back to the starting (aka Home) position.

## Turn Shape

### Default Appearance

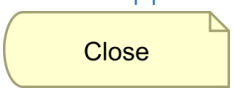



### What it Does

The Turn Shape changes the current direction of the "turtle" in the Turtle Graphics Window. Measurements are in degrees.

## Close Shape

### Default Appearance



### What it Does

The Close Shape is used with file I/O. After a file is first opened, to read or write, it must be later closed.

## Open Shape

### Default Appearance




### What it Does

The Open Shape is used with file I/O. Before a file can be read from or written to, it must be opened first.

## Read Shape

### Default Appearance




### What it Does

The Read Shape is used to retrieve a line of text from an opened file. The text is stored into a variable.

## Write Shape

### Default Appearance




### What it Does

The Write Shape evaluates an expression and then stores it into an opened file.

## Instructions / Guidelines / Description:

- Explore Flowgorithm's interface and understand how to create basic flowcharts using Assign, Input, Output, Decision, and Loop shapes.
- Practice declaring variables of different data types and using operators.
- Implement simple and available flowcharts
- Use appropriate data types and operators as described in the theory section.

## Applications:

List at least three applications of the concepts learned today, such as:

## Activity Questions:

1. What are the advantages of using Flowgorithm for learning programming?

2. How does Flowgorithm help in understanding the concept of data types?

3. Explain the difference between Integer and Real data types with examples.

4. What is the difference between the ‘Assignment’ shape and the ‘Call’ shape in Flowgorithm?

5. Describe the purpose of the ‘Breakpoint’ shape in Flowgorithm and how it assists in debugging.

.

6. Why is it important to understand data types when creating flowcharts in Flowgorithm?

7. Describe the function and usage of the ‘Input’ and ‘Output’ shapes in Flowgorithm with examples.

## Results/Conclusion:

**Date:** **Signature of Faculty**

# Experiment – 2
# Addition and subtraction of two numbers.

## Aim: To perform visualization of addition and subtraction of two numbers using flowgorithm.

## Tools required: Flowgorithm App.

## Theory:

Flowgorithm is a visual programming tool designed to help beginners and experienced programmers alike to develop algorithms using flowcharts. One of the fundamental aspects of programming is performing arithmetic operations. In this note, we will explore the different arithmetic operations that can be performed using Flowgorithm and how to implement them in your algorithms.

**Common Arithmetic Operations:**

1. **Addition (+):**

The addition operation is used to combine two or more numbers to obtain their sum. In Flowgorithm, you can use the "Process" symbol to add two or more variables and store the result in another variable.

2. **Subtraction (-):**

The subtraction operation is used to find the difference between two numbers. Similar to addition, you can use the "Process" symbol in Flowgorithm to subtract one variable from another and store the result.

In this Flowgorithm example, we will create a simple algorithm to add and subtract two numbers entered by the user. Flowgorithm is a visual programming tool that uses flowcharts to represent the logic of an algorithm. By following this note, you will learn how to use Flowgorithm to perform basic arithmetic operations and display the results.

## Step by step procedure:

Step 1: Start

Step 2: Input the first integer

Step 3: Input the second integer

Step 4: Add the two integers (Example: Sum = A + B)

Step 5: Display the sum

Step 6: Subtract the two integers (Example: Difference = A – B)

Step 7: Display the difference

Step 8: End

Step 9: Connect the symbols. Connect the symbols in the correct sequence by using arrows to show the flow of the algorithm.

Step 10: Run the algorithm

## Sample Output:

Enter first number: 10

Enter second number: 5

Sum = 15

Difference = 5

## Flowgorithm:

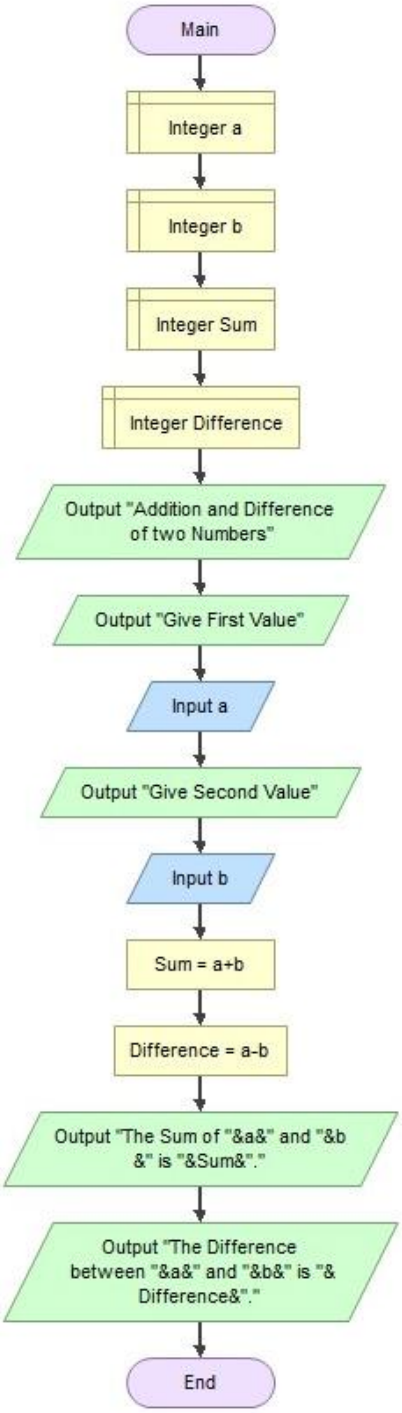



**Fig.1: Sample flowchart of basic addition of two numbers**

## Instructions / Guidelines / Description:

- Explore Flowgorithm's interface app and understand how to create basic flowcharts using Assign, Input, Output, Decision, and Loop shapes.
- Use the Declare shape to define four integer variables: a, b, sum, and difference.
- Use Input shapes to accept two numbers from the user.
- Use Process shapes to:

1. Add the two numbers and store the result in sum.
2. Subtract the second number from the first and store the result in difference.

- Use Output shapes to display the computed sum and difference.
- Run the flowchart and observe the output.
- Verify correctness using various inputs (positive, negative, zero).

## Applications:

Students are required to list at least three real-life applications of basic arithmetic operations, such as:

## Add your flowchart for basic addition of two numbers

## Add Results of basic addition of two numbers:

**Add your flowchart for basic subtraction of two numbers:**

## Add Results of basic subtraction of two numbers:

**Practice Question 1:** Create a Flowgorithm flowchart to calculate the total cost of two items. The program should:

- Prompt the user to enter the cost of the first item.
- Prompt the user to enter the cost of the second item.
- Calculate the total cost by adding both item costs.
- Display the total cost to the user.

## Results of Practice Question 1:

**Practice Question 2:** You are designing a simple billing system for a bookstore. Sometimes customers return a book, and the cost of that returned book must be subtracted from their original bill.

Create a Flowgorithm flowchart that:

1. Prompts the user to enter the original total bill amount.
2. Prompts the user to enter the cost of the returned book.
3. Calculate the updated total by subtracting the return amount from the original bill.
4. Displays the updated total bill to the user.

## Results of Practice Question 2:

## Activity Questions:

1. What is the purpose of the Process shape in Flowgorithm?

2. What happens if the user enters a negative number? How does Flowgorithm handle it?

3. Describe what would happen if the Input shape is skipped in the flowchart.

4. Write the flowchart to display the average of two numbers.

**Results/Conclusion:**

**Date:** **Signature of Faculty**

# Experiment – 3
# Addition and subtraction of two numbers using formatting methods

## Aim: To perform visualization of addition and subtraction using various formatting methods

## Tools required: Flowgorithm App.

## Theory:

Flowgorithm is a visual programming tool designed to help beginners and experienced programmers alike develop algorithms using flowcharts. It provides an intuitive way to visualize the flow of logic and data in a program, making it easier to understand programming concepts.

One fundamental programming task is performing arithmetic operations such as addition and subtraction. Flowgorithm allows you to perform these operations visually by using flowchart symbols that represent processes and data flow.

**Common Arithmetic Operations:**

**Addition (+):**
Addition calculates the total sum of two or more numbers. In Flowgorithm, you use the "Process" shape to add two variables and store the result in another variable.

**Subtraction (-):**
Subtraction calculates the difference between two numbers. Similarly, the "Process" shape subtracts one variable from another and stores the result.

### Formatting Methods with `Format` Command:

Flowgorithm provides a powerful **`Format`** command to control how output is displayed. Using `Format`, you can specify the number of decimal places, align text, and embed variable values inside formatted strings for clearer output.

For example, to display the sum of two numbers with exactly two decimal places, you can write:

```
Format("Sum = {0:0.00}", sum)
```

This formats the value of `sum` to two decimal places in the output.

Similarly, you can format subtraction results or combine text and numbers in a neat, readable way:

```
Format("Difference = {0:0.00}", difference)
```

Using `Format` helps produce professional-looking outputs and makes the results easier to interpret for the user.

*Special Note: Format command may not be supported by all flowgarithm apps.*

## Step-by-step procedure using Format command:

Step 1: Start
Step 2: Input the first integer and store it in variable A - *Use Input shape*
Step 3: Input the second integer and store it in variable B - *Use Input shape*
Step 4: Add the two integers
Sum = A + B - *Use Process shape*
Step 5: Display the sum using the Format command:
Format("Sum = {0}", Sum) - *Use Output shape*
Step 6: Subtract the two integers
Difference = A – B -*Use Process shape*
Step 7: Display the difference using the Format command:
Format("Difference = {0}", Difference) - *Use Output shape*
Step 8: End
Step 9: Connect the symbols in the correct sequence using arrows to indicate the flow of the algorithm.
Step 10: Run the algorithm.

## Sample Output:

Enter first number: 10

Enter second number: 5

Sum = 15

Difference = 5

## Instructions / Guidelines / Description:

- Explore Flowgorithm's interface app and understand how to create basic flowcharts using Assign, Input, Output, Decision, and Loop shapes.
- Use Flowgorithm to create a flowchart that accepts two integer inputs from the user.
- Perform addition and subtraction operations on these integers.
- Use the Format command to display the results neatly.
- Connect all flowchart symbols correctly to represent the flow of the algorithm.
- Run the program to test and verify the output.

## Applications:

Students are required to list at least three real-life applications of basic arithmetic operations, such as:

**Add your flowchart for addition of two numbers using formatting.**

**Add Results of addition of two numbers:**

**Add your flowchart for subtraction of two numbers using formatting:**

## Add Results of subtraction of two numbers:

**Practice Question 1:** You are helping a school develop a small program to determine the top scorer in a quiz competition. The scores of three students are entered into the system, and the program should find out who scored the highest.

Create a Flowgorithm flowchart that:

- Prompts the user to enter the scores of three students.
- Compares the scores to find the highest (largest) value.
- Displays the highest score as the top scorer's result.

## Result of Practice Question 1:

**Practice Question 2:** A parking lot charges a fixed fee for every full hour parked, but also needs to calculate if there's any extra time that should be billed separately.

Create a Flowgorithm flowchart that:

- Prompts the user to enter the total parking time in minutes.
- Calculates how many full hours the car was parked (using integer division).
- Calculates the remaining minutes after the full hours (using the mod operation).
- Displays the number of full hours and the remaining minutes to the user.

## Result of Practice Question 2:

## Activity Questions:

1. What is the purpose of the Format command in Flowgorithm?

2. What other methods can be used to perform and display the addition of two numbers in Flowgorithm without utilizing the Format command? Describe the logic behind these approaches.

## Results/Conclusion:

**Date:** **Signature of Faculty**

# Experiment – 4
# Calculation of area of circle and identification of odd even numbers

## Aim:

1. **To calculate the area of a circle given its radius.**
2. **To identify whether a number is odd or even.**

## Tools required: Flowgorithm App.

## Theory:

- **Calculation of Area of Circle:**
  The area of a circle is given by the formula:

$$Area = \pi \times r^2$$

where r is the radius of the circle and π is approximately 3.1416.

- **Identification of Odd and Even Numbers:**

An integer is called even if it is divisible by 2 without a remainder; otherwise, it is odd.

Even number: $n\%2 = 0$

Odd number: $n\%2 \neq 0$

## Step by Step Procedure:

- **Part 1: Flowchart to Calculate Area of Circle**

1. **Open Flowgorithm**

2. **Start a New Flowchart**

   **Add Input Symbol**

   - Drag and drop an **Input/Output** symbol (parallelogram) onto the workspace.
   - Double-click it and enter the prompt message:
     `Enter the radius of the circle:`
   - Create a variable to store the radius (e.g., `radius`) of type **Real** or **Float**.
3. **Add Assignment Symbol for Calculation**

- Drag an **Assignment** symbol (rectangle) below the input symbol.
- Double-click it and enter the formula to calculate the area:
  `area = 3.1416 * radius * radius`
- Create a variable named `area` of type **Real** or **Float** if prompted.

4. **Add Output Symbol**
   - Drag another **Input/Output** symbol below the assignment.
   - Double-click it and enter: `"Area of the circle = " + area`
   - This will display the calculated area.
5. **Connect the Symbols**
   Ensure all symbols are connected in sequence from Start → Input → Assignment → Output → End.
6. **Run and Test**

- **Part 2: Flowchart to Identify Odd or Even Number**

1. **Add Input Symbol**
   - After the area output, drag an **Input/Output** symbol to get an integer from the user.
   - Prompt message: `Enter an integer number:`
   - Create variable `num` of type **Integer**.
2. **Add Decision Symbol**
   - Drag a **Decision** symbol (diamond shape) below the input.
   - Double-click it and enter the condition: `num % 2 = 0`
   - This checks if the remainder when dividing by 2 is zero.
3. **Add Output for Even**
   - From the **True** branch of the decision, drag an **Output** symbol.
   - Message: `"The number " + num + " is Even."`
4. **Add Output for Odd**
   - From the **False** branch of the decision, drag another **Output** symbol.
   - Message: `"The number " + num + " is Odd."`
5. **Connect to End**
   - Connect both output symbols back to the **End** symbol.
6. **Run and Test**

## Sample Outputs:

- **Area Calculation Output:**

Enter radius: 5
Area of the circle = 78.54

- **Odd/Even Output:**

Enter number: 7
7 is Odd

## Instructions / Guidelines / Description:

- Explore Flowgorithm's interface app and understand how to create basic flowcharts using Assign, Input, Output, Decision, and Loop shapes, etc.
- Use Flowgorithm to create a flowchart that accepts two integer inputs from the user.
- Perform addition and subtraction operations on these integers.
- Use the Format command to display the results neatly.
- Connect all flowchart symbols correctly to represent the flow of the algorithm.
- Run the program to test and verify the output.

## Applications:

Students are required to list at least two real-life applications of basic area of circle and Even/Odd numbers, such as:

**Practice Question 1:** You want your flowchart to accept only positive numbers as the radius of the circle.

- How would you modify your flowchart to check if the radius entered is positive?
- What should the program do if the radius is zero or negative?
- Draw the flowchart to validate your inputs.

## Result of Practice Question 1:

**Practice Question 2:** If the user accidentally inputs a non-integer value when asked to check for odd or even, what problems could occur?

- How can your flowchart be improved to handle or prevent invalid inputs?
- Draw the flowchart to validate the same.

## Result of Practice Question 2:

## Activity Questions:

1. How did you implement the calculation of area in Flowgorithm?

2. How does the decision symbol work for checking odd or even?

3. What happens if you enter a negative radius? How can you improve your flowchart to handle such cases?

## Results/Conclusion:

**Date:** **Signature of Faculty**

# Experiment – 5
# Familiarization with Vectors and matrices

**Aim:** To understand and practice basic arithmetic operations on vectors and matrices using flowcharts in Flowgorithm.

## Tools required: Flowgorithm App.

## Theory:

### Vector (1D Array):

A **vector** is a collection of elements arranged in a single row or a single column.

A vector is usually represented as a **1D array**.

Example:

V=[2,4,6,8,10]

Here, V is a vector of size 5.

**In Flowgorithm:**

We declare it as an **array with one dimension**.

Example: `Integer V[5]`

We can access elements using an index: `V[0], V[1], … V[4].`

### Matrix (2D Array)

A **matrix** is a rectangular arrangement of numbers in **rows and columns**.

It is represented as a **2D array**.

Example:

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Here, M is a **3 × 3 matrix** (3 rows, 3 columns).

**In Flowgorithm:**

We declare it as a **two-dimensional array.**

Example: `Integer M[3][3]`

Elements are accessed by row and column indices: `M[0[0], M[0][1], M[1][2]`, etc.

**EXAMPLE CODE:**

Create a vector of 5 elements using flowgorithm.

Steps to create a vector of 5 elements in Flowgorithm:

1. **Start**: Begin your flowchart with the Start symbol.
2. **Declare a integer variable n to give the size of the vector**
3. **Declare a integer variable i to keep track of the indices of the vector**:
4. **Input Symbol**:
    - Use the **Input** symbol to prompt the user to enter the size of the vector.
    - Use the **Input** symbol to prompt the user to enter the initial index of the vector.
5. Introduce a while loop.
    - Enter element into each index of the vector.
    - Increment the index
    - Continue the loop until the condition satisfies.
6. **Process or Output**:
    - Add a **Output** block to display the vector entered.
7. **End**: Use the End symbol to finish the program.

**INSTRUCTIONS / GUIDELINES**

1. Design a Flowgorithm flowchart to:
    - Input a vector/matrix.
    - Use loops to take input and display the elements of the vector or matrix.
    - Apply arithmetic operations.
    - Display the output.
2. Run your flowchart with different vectors and dimension of matrices as inputs and note down outputs.
3. Save the flowchart file.
4. Take screenshots of your flowchart and output console; paste these into your notebook.

## Applications:

Students are required to list at least two applications on vectors and matrices such as:

**Practice Question 1:** Create a matrix of size "2×2"  and display using Flowgorithm.

## Result of Practice Question 1:

**Practice Question 2:** Enter two matrices A and B. Find the difference of two matrices and display using Flowgorithm.

## Result of Practice Question 2:

## Activity Questions:

1. What change do you recommend in your program to implement multiplication of two matrices?

2. How did you store the values of a vector?

3. What happens if you enter two matrices with different sizes in your flowgarithm.

## Results/Conclusion:

**Date:** **Signature of Faculty**

# Experiment – 6

## Solving Quadratic equations and generation of Fibonacci numbers

**Aim:** To familiarize with the solving quadratic equations and generate Fibonacci series using Flowgorithm.

## Tools required: Flowgorithm App.

**Theory:** A **quadratic equation** is a polynomial equation of degree 2. It is generally written as:

$$ax^2 + bx + c = 0$$

where a,b,c are real (or complex) numbers,

a≠0 (otherwise it becomes linear),

x is the unknown variable.

### 2. Standard Form

The standard quadratic equation is:

$$ax^2 + bx + c = 0$$

Here:

a = coefficient of $x^2$ (quadratic term)

b = coefficient of x (linear term)

c = constant term

### 3. Roots of a Quadratic Equation

The values of x that satisfy the quadratic equation are called its **roots (or solutions)**.

The formula for roots is given by the **quadratic formula**:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

### 4. Discriminant

The expression under the square root:

$$D = b^2 - 4ac$$

is called the **discriminant**.

If D>0: two distinct real roots.

If D=0: two equal real roots (repeated root).

If D<0: no real roots (two complex conjugate roots)

## Fibonacci Series:

### 1. Definition

The **Fibonacci series** is a sequence of numbers in which each number (called a **Fibonacci number**) is the sum of the two preceding ones.

The sequence starts with:

$$0,1,1,2,3,5,8,13,21,34,\ldots$$

### 2. Mathematical Representation

If $F_n$ represents the $n^{th}$ Fibonacci number, then:

$$F_n = F_{n-1} + F_{n-2}, \quad \text{for } n \geq 2$$

with the initial conditions:

$F_0=0$, $F_1=1$

**EXAMPLE CODE:**

Accept the quadratic and linear coefficients and constant term as the input from the user.

### Steps to find the roots of the quadratic equation in Flowgorithm:

1. **Start**: Begin your flowchart with the Start symbol.
2. **Declare three variables for the coefficients and two variables to store the roots of the equation**:
    - Use a **Declare** symbol to create a variable of type integer.
    - Example: Declare a as `integer`.
3. **Input Symbol**:
    - Use the **Input** symbol to prompt the user to enter the three variables.
    - In the Input box, write something like: `"Enter the quadratic coefficient"`
    - Store the input in the variable a.

**4. Process**:

- You can add a **Process** symbol to calculate the roots of the equation as per the formula.

5.Output:

- You can add a **output** symbol to display the roots of the equation.

**End**: Use the End symbol to finish the program.

## Applications:

Students are required to list at least two applications on Quadratic equations and generation of Fibonacci numbers:

**INSTRUCTIONS / GUIDELINES**

1. Design a Flowgorithm flowchart to:
   - Input a quadratic equation.
   - Display the roots of the equation.
2. Add comments in the flowchart blocks where possible.
3. Run your flowchart with different equations and note down outputs.
4. Save the flowchart file.
5. Take screenshots of your flowchart and output console; paste these into your notebook.

**Practice Question 1:** 1. Draw a flowchart to solve and find the roots of the quadratic equation using Flowgorithm.

## Result of Practice Question 1:

**Practice Question 2:** Draw a flowchart to generate a Fibonacci series of length 10 using Flowgorithm.

## Result of Practice Question 2:

## Activity Questions:

1. How did your program calculate the discriminant ($b^2 – 4ac$)?

2. How did your program generate the first two numbers of the Fibonacci series?

3. How does your program decide when to stop generating Fibonacci numbers?

4. What happens if the user enters a value less than 1? How can you improve your flowchart to handle such input?

## Results/Conclusion:

**Date:** **Signature of Faculty**

# Experiment – 7
# Modelling Simple resistive circuits

**Aim:** To analyze and model resistive circuits using Flogorithm.

## Tools required: Flowgorithm App.

## Theory:

1. **Ohm's Law**

Ohm's Law states that the current through a conductor between two points is directly proportional to the potential difference across the conductor, provided the temperature and other physical conditions remain constant.

$$V = I \times R$$

where,

V = Voltage (Volts)

I = Current (Amperes)

R = Resistance (Ohms, Ω)

This forms the basis for analyzing all resistive circuits.

2. **Resistances in Series**

When resistors are connected end-to-end, they are said to be in series. In this case, the same current flows through each resistor, and the total resistance is the sum of individual resistances:

$$R_{eq} = R_1 + R_2 + R_3 + \cdots + R_n$$

$$I = \frac{V}{R_{eq}}$$

Voltage division: $V_i = I \times R_i$

3. **Resistances in Parallel**

When resistors are connected such that both ends are joined to the same two points, they are in parallel. In this case, the same voltage is applied across each resistor, and the reciprocal of the equivalent resistance is the sum of the reciprocals of the individual resistances:

$$R_{eq} = \frac{1}{R_1} + \frac{1}{R_2} + \frac{1}{R_3} + \cdots + \frac{1}{R_n}$$

Voltage: V is the same across all branches.

Current division:

$$I_i = \frac{V}{R_i}$$

**EXAMPLE CODE:**

*Accept the values of the resistors as the input from the user.*

Steps to model resistive circuits in Flowgorithm:

1. **Start**: Begin your flowchart with the Start symbol.
2. **Declare the variables for the resistors**:
    - Use a **Declare** symbol to create a variable of type integer.
    - Example: Declare a as `integer`.
3. **Input Symbol**:
    - Use the **Input** symbol to prompt the user to enter the variable.
    - In the Input box, write something like: `"Enter the value of first resistor"`
    - Store the input in the variable a.
4. **Process**:
    - You can add a **Process** symbol to calculate equivalent resistance according to the corresponding formula for series and parallel.
5. Output:
    - You can add a **output** symbol to display the equivalent resistance.
6. **End**: Use the End symbol to finish the program.

## Applications:

Students are required to list at least two applications on use of resistors in real world (with necessary diagram).

**INSTRUCTIONS / GUIDELINES**

1. Design a Flowgorithm flowchart to:
    - Input the values of the resistors.
    - Display the equivalent resistance of the circuits.
2. Add comments in the flowchart blocks where possible.
3. Run your flowchart with different equations and note down outputs.
4. Save the flowchart file.
5. Take screenshots of your flowchart and output console; paste these into your notebook.

**Practice Question 1:** Draw a flowchart to solve and find the equivalent resistance of resistors arranged in series using Flowgorithm.

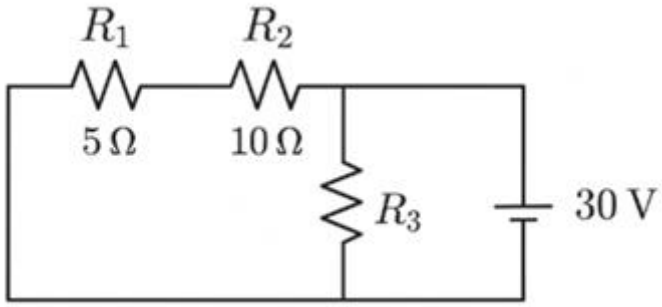
## Result of Practice Question 1:

**Practice Question 2:** Draw a flowchart to solve and find the equivalent resistance of resistors arranged in parallel using Flowgorithm.

## Result of Practice Question 2:

## Activity Questions:

1. Calculate R3 in the circuit given below using Flowgorithm ?




2. How does resistance affect the performance or safety of the device?

## Results/Conclusion:

**Date:** **Signature of Faculty**

# Experiment – 8
# Familiarization with Arrays to solve problems

**Aim:** To understand and practice to solve problems using array.

## Tools required: Flowgorithm App.

## Theory:

An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations**, accessed by a single name and an index (or subscript).

Integer marks[5] = {90, 85, 80, 95, 88};

Here marks is an array of 5 integers.

### Types of Arrays:

### One-Dimensional Array

1. Linear collection of elements.

   Example: `Integer a[10];`

**Two-Dimensional Array**

1. Like a table with rows and columns.

   Example: `Integer matrix[3][4];`

**EXAMPLE CODE:**

Create an array of 2X2 dimension using flowgorithm.

### Steps to create an array in Flowgorithm:

1. **Start**: Begin your flowchart with the Start symbol.
2. **Declare** integer variables row, column, to initialize the dimension of the array.
3. **Declare** integer variables i and j to keep track of the indices of the rows and column of the array.
4. **Input Symbol**:

   - Use the **Input** symbol to prompt the user to enter the dimension of the array.
   - Use the **Input** symbol to prompt the user to enter the initial index of the vector.

5. Introduce a two for loops: one outer for loop for rows and one inner for loop for columns.

- Enter element into each index of the array.
- Increment the index of the row and column respectively
- Continue the loop until the condition satisfies.

6. **Process or Output**:

- Add a **Output** block to display the array entered.

7. **End**: Use the End symbol to finish the program.

## Applications:

Students are required to list at least two applications on use of arrays in real world.

**INSTRUCTIONS / GUIDELINES**

1. Design a Flowgorithm flowchart to:

- Input an array.
- Use loops to take input and display the elements of the arrayx.
- Display the output.

2. Run your flowchart with different arrays and dimension of arrays as inputs and note down outputs.
3. Save the flowchart file.
4. Take screenshots of your flowchart and output console; paste these into your notebook.

**Practice Question 1:** Draw a flowchart to find the sum of all the array elements in Flowgorithm.

## Result of Practice Question 1:

**Practice Question 2:** Draw a flowchart to find an 1D array and find out the largest element in Flowgorithm.

## Result of Practice Question 2:

## Activity Questions:

1. What happens if you try to input more elements than the declared array size?

2. How did your program determine the largest value in the array?

3. How does your program check each array element to determine if it is even or odd?

4. How does your program calculate the average of the array elements?

## Results/Conclusion:

**Date:** **Signature of Faculty**

# Experiment – 8
# Familiarization with strings

**Aim:** To understand and practice solving problems using strings in Flowgorithm.

## Tools required: Flowgorithm App.

## Theory:

A string is a sequence of characters used to represent text in programming. Strings can include letters, numbers, symbols, and spaces, and are typically enclosed within quotation marks. String manipulation involves performing operations such as accessing individual characters, extracting substrings, concatenation (joining strings), and modifying or analysing the content. Understanding how to work with strings is fundamental because text data is pervasive in programming—from user input and file processing to displaying messages and formatting outputs.

In flowchart-based programming tools like Flowgorithm, strings are handled using specific functions such as Len (to find the length), etc. Using loops and these functions, various string operations like reversing a string or searching for a substring can be visualized and implemented step-by-step. This experiment aims to familiarize students with basic string operations and how to represent the logic clearly through flowcharts, which aids in better understanding and debugging of programs involving strings.

**EXAMPLE CODE:**

### Steps to Work with Strings in Flowgorithm:

1. **Start:**
   Begin with the Start symbol.
2. **Declare Variables:**
   - Declare a string variable (e.g., `text`).
   - Declare integer variables (e.g., `i`, `count`) for indexing and counting.
3. **Input Symbol:**
   - Prompt the user to enter a string.
     Example: *"Enter a string:"*
4. **Process Symbol / Loops:**
   - Use a loop to examine each character of the string.

- Check for conditions such as vowel, space, matching character, etc.

5. **Output Symbol:**
   - Display results such as length, vowel count, reversed string, etc.
6. **End:**
   Complete the flowchart using the End symbol.

## Applications:

Students are required to list at least two real-world applications involving strings.

**INSTRUCTIONS / GUIDELINES**

1. Design a Flowgorithm flowchart to:

   - Input a string
   - Use loops to process the string (e.g., count characters)
   - Display the result

2. Run the flowchart with different string inputs (sentences, words, numeric strings) and note down the outputs.
3. Save your Flowgorithm file.
4. Take screenshots of your flowchart and output console; paste them into your notebook.

**Practice Question 1:** Draw a flowchart to display the length of a string using flowgarithm.

## Result of Practice Question 1:

**Practice Question 2:** Draw a flowchart to perform concatenation of two strings using flowgarithm.

## Result of Practice Question 2:

## Activity Questions:

1. What is the length of the string "Flowgorithm"?

2. What are the first and last characters of the string "Experiment"?

3. What is the ASCII value of the character 'Z'?

4. What is the result when you concatenate "Good" and "Morning"?

## Results/Conclusion:

**Date:** **Signature of Faculty**

## Experiment – 9

## Familiarization with Searching (Linear & Binary) Using Flowgorithm

**Aim:** To implement Linear Search and Binary Search algorithms using Flowgorithm.

**Tools required: Flowgorithm App.**

**Theory:**

### Linear Search

Linear Search is a straightforward searching algorithm that works by checking each element of a list one by one until the desired element is found or the end of the list is reached. It does not require the data to be sorted, making it suitable for unsorted arrays or lists. Although easy to implement, Linear Search is not efficient for large datasets as its time complexity is O(n) in the worst case, meaning it may have to check every element before finding the target or determining it is not present.

### Binary Search

Binary Search is an efficient algorithm used to find a target element in a sorted array. It works by repeatedly dividing the search range in half and comparing the middle element with the target value. If the middle element matches the target, the search is successful. Otherwise, the search continues in either the left or right half, depending on whether the target is smaller or larger than the middle value. This divide-and-conquer approach significantly reduces the number of comparisons, giving Binary Search a time complexity of O (log n) in the worst case.

## Step by Step Procedure:

*Perform a linear search on an array of 5 integers.*

1. Start the program.
2. Create an array to hold 5 numbers.
3. Ask the user to enter 5 numbers one by one and store them in the array.
4. Ask the user to enter the number to search.
5. Set a variable called Flag to 0 (means not found yet).
6. Go through each number in the array one by one:
7. If the number matches the search number:
    a. Set Flag to 1.
    b. Remember the position where it was found.
    c. Stop checking more numbers.
8. If found is 1, show “Number found at position X”.
9. Else, show “Number not found”.
10. End the program.

## Example Code in Flowgorithm Pseudocode:

## Linear Search:

```
Start
  Declare arr[5] = {10, 20, 30, 40, 50}
  Declare target, index As Integer
  Declare found As Boolean = False

  Input "Enter element to search: " → target

  For index = 0 To 4
    If arr[index] = target Then
      found = True
      Output "Element found at position: " + (index + 1)
      Break
    End If
  End For

  If found = False Then
    Output "Element not found in the array."
  End If
End
```

## Binary Search:

```
Start
  Declare arr[5] = {10, 20, 30, 40, 50}  // Sorted Array
  Declare target, low, high, mid As Integer
  Declare found As Boolean = False

  Input "Enter element to search: " → target

  low = 0
  high = 4

  While low <= high
    mid = (low + high) / 2
    If arr[mid] = target Then
      found = True
      Output "Element found at position: " + (mid + 1)
      Break
    Else If arr[mid] < target Then
      low = mid + 1
    Else
      high = mid - 1
    End If
  End While

  If found = False Then
    Output "Element not found in the array."
  End If
End
```

## Applications:

Students are required to list at least two real-life applications of Searching (Linear & Binary):

## Instructions / Guidelines / Description:

- Use Flowgorithm to create flowcharts for both Linear and Binary Search algorithms.
- Input arrays can be predefined or entered by the user.
- Test the flowcharts with various input elements, including elements present and absent in the array.
- Observe and compare how many steps each algorithm takes to find or fail to find the target.
- Note that Binary Search requires the array to be sorted.

**Practice Question 1:** Draw a flowchart to implement Linear Search in Flowgorithm.

## Result of Practice Question 1:

**Practice Question 2:** Draw a flowchart to implement **Binary Search** in Flowgorithm.

## Result of Practice Question 2:

## Activity Questions:

1. What are the main differences between Linear and Binary Search?

2. Why does Binary Search require a sorted array?

3. Which algorithm is more efficient for large datasets? Explain.

## Results/Conclusion:

**Date:** **Signature of Faculty**

# Experiment – 10

# Familiarization with Sorting (Bubble, Insertion, Selection) Using Flowgorithm

**Aim:** To understand and practice implementing sorting algorithms (Bubble Sort, Insertion Sort, and Selection Sort) using Flowgorithm.

## Tools required: Flowgorithm App.

## Theory:

Sorting is the process of arranging data in a specific order, usually ascending or descending, to make searching and processing easier. Bubble Sort, Insertion Sort, and Selection Sort are simple comparison-based algorithms often taught as a foundation in computer science. In Bubble Sort, adjacent elements are repeatedly compared and swapped if they are in the wrong order, causing larger values to "bubble up" to the end of the list. Although easy to understand, Bubble Sort is inefficient for large datasets because it requires many passes through the list.

In Selection Sort, the algorithm repeatedly selects the smallest (or largest) element from the unsorted part of the list and places it at the correct position. It reduces the number of swaps but still has a quadratic time complexity. Insertion Sort works more like sorting playing cards — elements are picked one by one and inserted into their correct position in the already sorted portion of the list. It performs well on small or nearly sorted datasets. While all three algorithms have O(n^2) worst-case complexity, Insertion Sort is generally more efficient than Bubble and Selection Sort in practical scenarios.

## Step by Step Procedure:

- *Start:*

*Begin flowchart with Start symbol.*

- *Declare Variables:*

*Array variable (e.g., arr[])*

*Integer variables for size, i, j, temp*

- *Input Symbols:*

*Ask user for array size*

*Use loops to input array elements*

• *Processing (Loops & Conditions):*

*Implement Bubble, Selection, or Insertion sorting logic*

*Use nested loops where required*

*Perform comparisons and swaps*

• *Output Symbol:*

*Display array after sorting*

• *End:*

## Applications:

Students are required to list at least two real-life applications of Sorting (Bubble, Insertion, Selection).

## Instructions / Guidelines / Description:

1. Sorting is the process of arranging data in a particular order (ascending or descending).
2. Identify the data elements to be sorted (e.g., numbers, names, or paired data like name–age).
3. Choose a sorting technique (e.g., Bubble Sort, Insertion Sort, Selection Sort).
4. Compare elements based on the sorting condition.
5. Swap elements if they are not in the correct order.
6. Repeat the process until all elements are arranged properly.
7. Display the sorted output clearly.
8. Verify correctness by checking if each element follows the required order.

**Practice Question 1:** Draw a flowchart to implement **Selection Sort** in Flowgorithm.

## Result of Practice Question 1:

**Practice Question 2:** Draw a flowchart to implement **Insertion Sort** in Flowgorithm.

## Result of Practice Question 2:

**Practice Question 3:** Draw a flowchart to implement **Bubble Sort** in Flowgorithm.

## Result of Practice Question 3:

## Activity Questions:

1. How did your program compare and swap elements during Bubble Sort?

2. How did your program identify the smallest element in Selection Sort?

3. How does your program shift elements while performing Insertion Sort?

## Results/Conclusion:

**Date:** **Signature of Faculty**

# Experiment – 11

## Modelling Circuits with Dependent Sources Using Flowgorithm

**Aim:** To understand and practice implementing sorting algorithms (Bubble Sort, Insertion Sort, and Selection Sort) using Flowgorithm.

## Tools required: Flowgorithm App.

## Theory:

In electrical circuits, a dependent source is a voltage or current source whose value depends on some other voltage or current in the circuit.

There are four types of dependent sources:

- Voltage-Controlled Voltage Source (VCVS)
- Voltage-Controlled Current Source (VCCS)
- Current-Controlled Voltage Source (CCVS)
- Current-Controlled Current Source (CCCS)

### *i. (i)Voltage-Controlled Voltage Source (VCVS)*

A VCVS is a voltage source whose output voltage depends on another voltage in the circuit. It is usually defined as:

$$Vout = A \times V_x,$$

where:

- $Vout$ is the voltage of the dependent source,
- $A$ is a constant (gain),
- $V_x$ is the controlling voltage somewhere else in the circuit.

### *ii. Current-Controlled Voltage Source (CCVS)*

A CCVS is a voltage source whose output depends on a current elsewhere in the circuit. It is mathematically represented as:

$$Vout = R \times I_x,$$

where:

- $Vout$ = output voltage of the dependent source
- $I_x$ = the controlling current in another part of the circuit
- $R$ = gain or transresistance (measured in ohms, Ω)

### *iii. Voltage-Controlled Current Source (VCCS)*

A VCCS is a current source whose output current depends on a voltage elsewhere in the circuit.

$$Iout = G \times V_x,$$

where:

- $Iout$ = Output current (A)
- $V_x$ = Controlling voltage (V)
- $G$ = Gain (conductance in siemens, S)

### *iv. Current-Controlled Current Source (CCCS)*

A CCCS is a current source whose output depends on a current elsewhere in the circuit.

$$Iout = B \times I_x,$$ where:

- *Iout = Output current (A)*
- $I_x$ *= Controlling current (A)*
- *B = Gain (unitless multiplier)*

## Step by Step Procedure:

### Steps to Model Dependent Sources in Flowgorithm:

1. **Start:**

   Start the flowchart with the Start symbol.
2. **Declare Variables:**

   Declare input variables (e.g., `Vx`, `Ix`, `etc.`).

   Declare output variables (`Vo`, `Io`).
3. **Input Symbols:**

   Prompt user to enter:
   - Controlling voltage or current
   - Gain value (μ, β, g, r depending on source type)
4. **Process Symbols:**

   Use the dependent source formula to compute the output.
5. **Output Symbols:**

   Display the calculated dependent source value.
6. **End:**

   Use the End symbol to finish the program.

## Applications:

Students must list at least two real-world applications where dependent sources are used.

## Instructions / Guidelines / Description:

1. Design a Flowgorithm flowchart to:
    - Input controlling voltage or current
    - Input the gain (μ, β, etc)
    - Compute and display the dependent voltage/current source
2. Run your flowchart with:
    - Different gain values
    - Different controlling inputs
    - Different dependent source types
3. Save your Flowgorithm file.
4. Take screenshots of your flowchart and output console and paste them into your notebook.

**Practice Question 1:** Draw a flowchart to compute the output voltage of a VCVS using flowgarithm.

## Result of Practice Question 1:

**Practice Question 2:** Draw a flowchart to compute the output current of a CCCS using flowgarithm.

## Result of Practice Question 2:

## Activity Questions:

1. How does a dependent source differ from an independent source in terms of circuit behavior?

2. Why must the controlling variable (voltage or current) be correctly identified before applying a dependent source model?

3. Explain how dependent sources affect the number of independent KCL/KVL equations required to solve a circuit.

## Results/Conclusion:

**Date:** **Signature of Faculty**